

Question No:2

DS

Session

Stack

Question Information

Level 1

Challenge 42

Question description

Rajinikanth organized a technical round interview in an Animation company for the set of computer science candidates.

the task is to implement stack operations for two stacks and merge the stacks into one.

Get two sets of stack elements and reverse them, then merge them into one stack.

Rajinikanth has given the deadline of only 15 minutes to complete the problem.

Can you Help the candidates to complete the problem within the specified time limit?

Function Description

a) push(): Adds the new item at the beginning of the linked list using the first pointer.

b) pop(): Removes an item from the beginning using the first pointer.

c) merge(): Links the first pointer second stack as next of the last pointer of the first list.

Constraints

$0 < n, m < N$

$1 < arr[i] < 1000$

Input Format:

First-line indicates n & m, where n is the number of elements to be pushed into the stack and m is the number of pop operations that need to be performed

The next line indicates the n number stack elements

Output Format:

First-line indicates the top of the element of the stack

the second line indicates the top of the element after the pop operation

Logical Test Cases

Test Case 1

INPUT (STDIN)

5 8
9 8 7 6 5
5 6 7 8 3 11 2 3

EXPECTED OUTPUT

5 6 7 8 9 3 2 11 3 8 7 6 5

Test Case 2

INPUT (STDIN)

10 4
91 18 17 16 15 15 16 17 18 13
11 12 13 14

EXPECTED OUTPUT

13 18 17 16 15 15 16 17 18 91 14 13 12 11

Mandatory Test Cases

Test Case 1

KEYWORD

void push(int data, mystack* ms)

Test Case 2

KEYWORD

int pop(mystack* ms)

Test Case 3

KEYWORD

void merge(mystack* ms1, mystack* ms2)

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

1

Test Case 2

TOKEN COUNT

435

Test Case 3

NLOC

90

Code:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct Node
5  {
6      int data;
7      struct Node *next;
8  };
9
10 void push(struct Node **top, int value)
11 {
12     struct Node *newNode;
13
14     newNode = (struct Node *)malloc(sizeof(struct Node));
15
16     newNode->data = value;
17     newNode->next = *top;
18     *top = newNode;
19 }
20
21 void merge(struct Node *top1, struct Node *top2)
22 {
23     struct Node *temp;
24
25     if (top1 == NULL)
26         return;
27
28     temp = top1;
29
30     while (temp->next != NULL)
```

```

31     {
32         temp = temp->next;
33     }
34
35     temp->next = top2;
36 }
37 void display(struct Node *top)
38 {
39     struct Node *temp = top;
40
41     while (temp != NULL)
42     {
43         printf("%d ", temp->data);
44         temp = temp->next;
45     }
46 }
47 int main()
48 {
49     int n, m, value;
50     struct Node *top1 = NULL;
51     struct Node *top2 = NULL;
52
53     scanf("%d %d", &n, &m);
54     /* First stack */
55     for (int i = 0; i < n; i++)
56     {
57         scanf("%d", &value);
58         push(&top1, value);

```

```
59     }
60     /* Second stack */
61     for (int i = 0; i < m; i++)
62     {
63         scanf("%d", &value);
64         push(&top2, value);
65     }
66     /* Merge */
67     merge(top1, top2);
68     /* Display */
69     display(top1);
70     return 0;
71 }
```

Output:

Test Case-1

	Output	
	<pre>5 8 9 8 7 6 5 5 6 7 8 3 11 2 3 5 6 7 8 9 3 2 11 3 8 7 6 5 === Code Execution Successful ===</pre>	

Test Case-2

	Output	
	<pre>10 4 91 18 17 16 15 15 16 17 18 13 11 12 13 14 13 18 17 16 15 15 16 17 18 91 14 13 12 11 === Code Execution Successful ===</pre>	

Keshav Kumar Rauniyar
CH.SC.U4CSE25236